AEC2 - HDFS

Etiquetas

Introducción

Esta práctica la he desarrollado en tres bloques: primero trabajo con HDFS y un flujo de comandos completo (incluyendo fallos típicos y su corrección). Luego hago una representación manual de MapReduce con registros simulados. Por último, cierro con un análisis de un sistema real bajo el teorema CAP.

Como extra he decidido añadir un script con un mini menú interactivo que ejecuta el bloque de HDFS y sirve de visor para los otros bloques, lo que me ha ayudado a depurar y a comprobar todo más rápido.

Bloque 1 - HDFS

Aquí planteo un caso sencillo sobre una editorial ficticia que quiere archivar y publicar libros en un repositorio distribuido. El flujo que he creado sigue esta forma: arranque de HDFS, comprobaciones, creación de directorio, subida de un libro, lectura parcial para verificar contenido, búsqueda y medición de espacio, y el cierre con renombrado, descarga y borrado.

Además incluyo tres errores concretos que me han parecido interesantes:

- subir un fichero local que no existe,
- listar una ruta de HDFS mal escrita,
- leer un fichero antes de subirlo (o con un nombre incorrecto).

Me parece una elección razonable porque son fallos muy habituales que a mí me han ocurrido en HDFS, y quería plasmar cómo se detectan y cómo se corrigen.

Comandos completos (bloque 1)

Ruta de Hadoop usada en los apuntes: `/home/bigdata/hadoop-3.3.6/`.

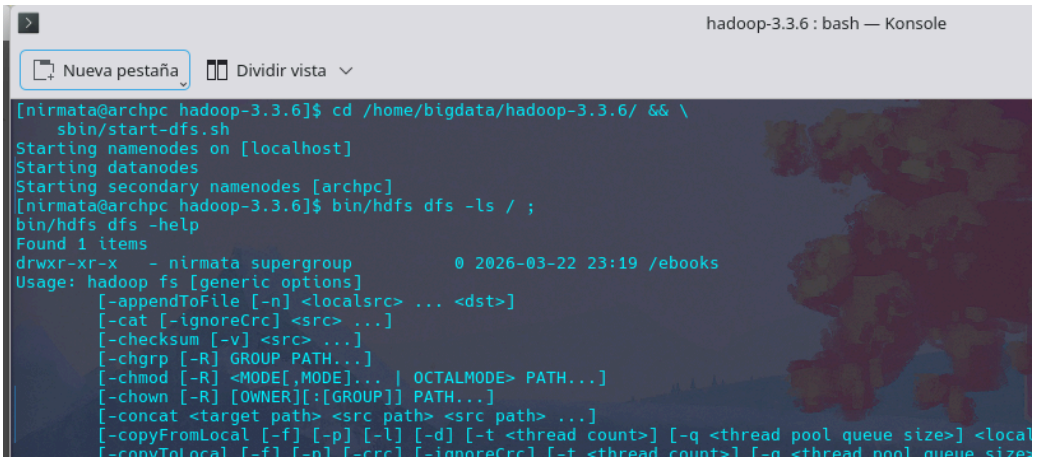
Arrancar Hadoop y HDFS (en un solo bloque relacionado)

```
cd /home/bigdata/hadoop-3.3.6/ && \
    sbin/start-dfs.sh
```

Comprobaciones iniciales (lista raiz y ayuda)

```
bin/hdfs dfs -ls / ;
```

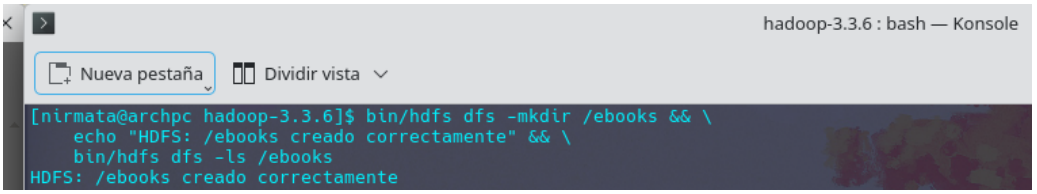
```
bin/hdfs dfs -help
```



Inicio de pruebas

Crear directorio de trabajo y verificar su creación

```
bin/hdfs dfs -mkdir /ebooks && \
    echo "HDFS: /ebooks creado correctamente" && \
    bin/hdfs dfs -ls /ebooks
```

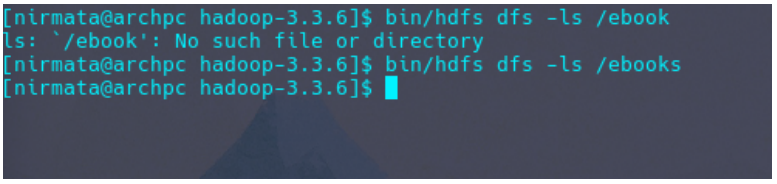


Error intencionado 1: ruta mal escrita

```
bin/hdfs dfs -ls /ebook
```

Correccion: ruta correcta

```
bin/hdfs dfs -ls /ebooks
```



Preparar carpeta local y descargar libro (bloque relacionado)

```
mkdir -p /home/bigdata/datos && \
wget -q https://www.gutenberg.org/cache/epub/11/pg11.txt -O /home/bigdata/
```

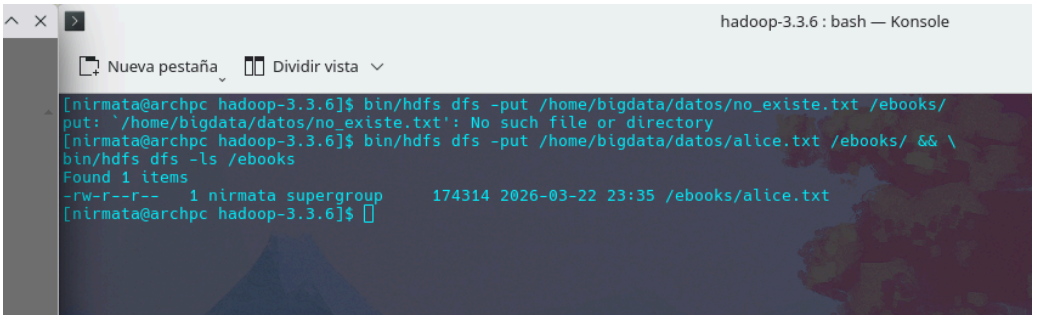
```
datos/alice.txt && \
echo "Ficheros en /home/bigdata/datos:" && ls -l /home/bigdata/datos
```

Error intencionado 2: fichero local no existe

```
bin/hdfs dfs -put /home/bigdata/datos/no_existe.txt /ebooks/
```

Correccion: subir el archivo correcto y verificar subida

```
bin/hdfs dfs -put /home/bigdata/datos/alice.txt /ebooks/ && \
bin/hdfs dfs -ls /ebooks
```



Error intencionado 3: leer antes de tiempo / nombre incorrecto

```
bin/hdfs dfs -head /ebooks/alice_no_subido.txt
```

Correccion: leer el fichero correcto y comprobar primeras líneas

```
bin/hdfs dfs -head /ebooks/alice.txt
```

Otras pruebas

Comprobar contenido en HDFS

```
bin/hdfs dfs -ls /ebooks
```

```
[nirmata@archpc hadoop-3.3.6]$ bin/hdfs dfs -ls /ebooks
Found 1 items
-rw-r--r--   1 nirmata supergroup     174314 2026-03-22 23:35 /ebooks/alice.txt
[nirmata@archpc hadoop-3.3.6]$
```

Ver final del fichero

```
bin/hdfs dfs -tail /ebooks/alice.txt
```

```
[nirmata@archpc hadoop-3.3.6]$ bin/hdfs dfs -tail /ebooks/alice.txt
se visit: www.gutenberg.org/donate.

Section 5. General Information About Project Gutenberg electronic works

Professor Michael S. Hart was the originator of the Project
Gutenberg concept of a library of electronic works that could be
freely shared with anyone. For forty years, he produced and
distributed Project Gutenberg eBooks with only a loose network of
volunteer support.

Project Gutenberg eBooks are often created from several printed
editions, all of which are confirmed as not protected by copyright in
the U.S. unless a copyright notice is included. Thus, we do not
necessarily keep eBooks in compliance with any particular paper
edition.

Most people start at our website which has the main PG search
facility: www.gutenberg.org.

This website includes information about Project Gutenberg,
including how to make donations to the Project Gutenberg Literary
Archive Foundation, how to help produce our new eBooks, and how to
subscribe to our email newsletter to hear about new eBooks.


[nirmata@archpc hadoop-3.3.6]$
```

Buscar el fichero

```
bin/hdfs dfs -find / -name alice.txt
```

```
[nirmata@archpc hadoop-3.3.6]$ bin/hdfs dfs -find / -name alice.txt
/ebooks/alice.txt
[nirmata@archpc hadoop-3.3.6]$
```

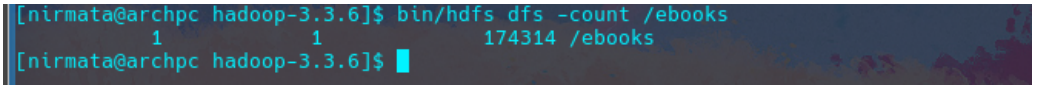
Medir uso de espacio (du)

```
bin/hdfs dfs -du -h /ebooks
```

```
[nirmata@archpc hadoop-3.3.6]$ bin/hdfs dfs -du -h /ebooks
170.2 K  170.2 K  /ebooks/alice.txt
[nirmata@archpc hadoop-3.3.6]$
```

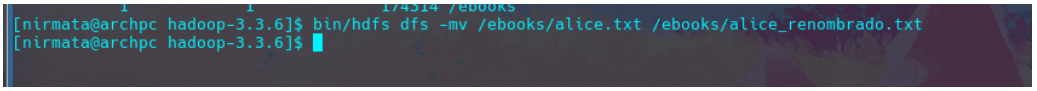
Medir uso de espacio (count)

```
bin/hdfs dfs -count /ebooks
```



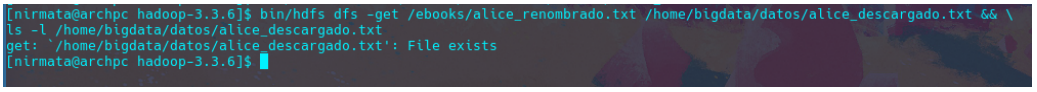
Renombrar

```
bin/hdfs dfs -mv /ebooks/alice.txt /ebooks/alice_renombrado.txt
```



Descargar a local y listar el fichero descargado

```
bin/hdfs dfs -get /ebooks/alice_renombrado.txt /home/bigdata/datos/alice_d
escargado.txt && \
ls -l /home/bigdata/datos/alice_descargado.txt
```

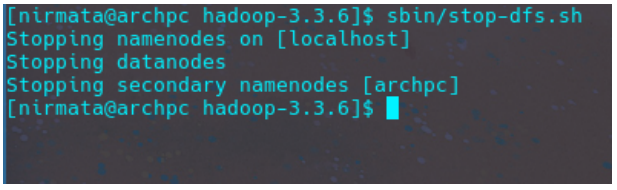


Borrar en HDFS

```
bin/hdfs dfs -rm /ebooks/alice_renombrado.txt
```

Parar HDFS

```
sbin/stop-dfs.sh
```



Bloque 2 - MapReduce

En este bloque quería dejar muy claro el flujo de MapReduce sin depender de un cluster, así que lo hice a mano con pocos registros y un criterio de negocio sencillo. La idea es que se vea la lógica completa desde la entrada hasta el porcentaje final. Lo traté casi como si fuera un experimento

controlado: pocos datos, un criterio bien definido y un resultado fácil de comprobar, para que el foco fuese entender el proceso y no pelearme con la infraestructura.

Escenario real elegido

He elegido el caso de uso de bicicletas compartidas en una gran ciudad. Lo que me interesa medir es el porcentaje de devoluciones tardías por estación, porque es un indicador muy útil para detectar puntos con problemas de disponibilidad o de funcionamiento. Es un ejemplo que se entiende rápido y, a la vez, se parece mucho a problemas reales de analítica operativa.

Justificacion Big Data

En mi opinión encaja en Big Data por varias razones. El volumen es alto porque hay millones de viajes al mes, cada uno con muchos atributos. La velocidad también es elevada porque los eventos se generan en tiempo real desde docks y apps. La variedad aparece en la mezcla de datos de viajes, logs de la aplicación y variables externas como el clima. La veracidad no es perfecta por fallos de sensores o usuarios, y el valor está en optimizar la operativa y mejorar la experiencia del usuario. Este resumen me sirve para justificar por qué MapReduce es una herramienta razonable en este tipo de casos.

Representación manual de Map y Reduce (10 registros)

Registros de entrada (simulados)

Formato: `(viaje_id, estacion, minutos_retraso)`

1. (v001, SOL, 0)
2. (v002, SOL, 7)
3. (v003, ATO, 0)
4. (v004, ATO, 12)
5. (v005, ATO, 4)
6. (v006, GRA, 0)
7. (v007, GRA, 0)
8. (v008, GRA, 9)
9. (v009, SOL, 0)
10. (v010, SOL, 3)

Criterio: considero tardía si `minutos_retraso > 5`. Esto simplifica el problema a una regla clara que luego puedo transformar fácilmente en clave-valor.

Fase Map

Emito `(estacion, (tardia, total))` para cada registro. Con esto dejo preparado el agregado final sin perder el detalle de cada viaje.

- v001 → (SOL, (0, 1))
- v002 → (SOL, (1, 1))
- v003 → (ATO, (0, 1))
- v004 → (ATO, (1, 1))

- v005 → (ATO, (0, 1))
- v006 → (GRA, (0, 1))
- v007 → (GRA, (0, 1))
- v008 → (GRA, (1, 1))
- v009 → (SOL, (0, 1))
- v010 → (SOL, (0, 1))

Shuffle/Sort

Agrupo por clave: aquí es donde se ve por qué MapReduce funciona tan bien cuando el problema es sumar o contar por categorías.

- SOL → (0,1), (1,1), (0,1), (0,1)
- ATO → (0,1), (1,1), (0,1)
- GRA → (0,1), (0,1), (1,1)

Fase Reduce

Sumo tardías y totales y calculo el porcentaje: en este punto el problema ya está resumido y se puede comparar estaciones de forma directa.

- SOL → tardias=1, total=4, porcentaje=25%
- ATO → tardias=1, total=3, porcentaje=33.3%
- GRA → tardias=1, total=3, porcentaje=33.3%

Resultado final: ATO y GRA presentan más retraso relativo que SOL. En un caso real, este resultado serviría para priorizar mantenimiento o ajustar la distribución de bicicletas.

Bloque 3 - Caso CAP

Para cerrar la práctica, preferí aterrizar CAP en un sistema real en lugar de quedarme en definiciones. Elegí Cassandra porque lo he visto en ejemplos de producción, porque es un clásico en bases de datos distribuidas y, además, encaja con lo que vimos en la asignatura de Bases de Datos Avanzadas en la unidad de bases de datos distribuidas donde se explican los compromisos entre disponibilidad, particiones y consistencia.

Sistema distribuido real: Apache Cassandra

He escogido Apache Cassandra porque es una base de datos distribuida muy usada en servicios con mucha carga (telemetría, logs, IoT). Me parece un ejemplo claro para mostrar los compromisos del teorema CAP. Además, su modelo maestro-less y su enfoque en disponibilidad permiten ver de forma práctica por qué en entornos reales no siempre se puede aspirar a la consistencia fuerte.

Analisis CAP

- Tolerancia a particiones (P): si hay cortes de red, el sistema sigue funcionando con los nodos disponibles.
- Disponibilidad (A): prioriza responder peticiones aunque exista particion, devolviendo respuestas locales.

- Consistencia (C): no siempre es estricta; se trabaja con consistencia eventual o configurada por nivel.

Justificación técnica

Cassandra replica datos y permite elegir niveles de consistencia en lectura y escritura. En caso de partición, prefiere seguir aceptando operaciones para no frenar el servicio (A) y tolerar la partición (P), aunque eso implique divergencias temporales entre réplicas. Esto me parece clave porque conecta la teoría con la operativa real: no se trata de que el sistema sea “menos correcto”, sino de que prioriza el servicio y luego repara el estado cuando la red se estabiliza.

Justificación de negocio

En sistemas de monitorización y logs veo más importante mantener el servicio activo y no perder datos que bloquear todo por una consistencia estricta. La consistencia eventual es suficiente porque en la mayoría de los casos creo que se puede reconciliar más tarde. En un contexto de negocio, ese tiempo de respuesta continuo vale más que la precisión inmediata en cada lectura, y por eso Cassandra resulta una elección coherente para este tipo de escenarios.

Referencias

- https://hadoop.apache.org/docs/current/hadoop-project-dist/hadoop-common/FileSystemShell.html (guía oficial de comandos `hadoop fs` y `hdfs dfs`)
- https://hadoop.apache.org/docs/current/hadoop-mapreduce-client/hadoop-mapreduce-client-core/MapReduceTutorial.html (tutorial oficial de MapReduce)
- https://cassandra.apache.org/doc/latest/ (documentación oficial de Cassandra)
- https://cassandra.apache.org/_/quickstart.html (quickstart oficial con Docker y CQL)
- https://www.ibm.com/think/topics/cap-theorem (explicación clara del teorema CAP con ejemplos)
- https://en.wikipedia.org/wiki/CAP_theorem (resumen general y referencias académicas)